Lab 6: Virtual Memory

Contents

- Introduction
- Goals of this lab
- Background
- Basic Exercises
- Advanced Exercises

Introduction

Virtual memory provides isolated address spaces, so each user process can run in its address space without interfering with others.

In this lab, you need to initialize the memory management unit(MMU) and set up the address spaces for the kernel and user processes to achieve process isolation

Goals of this lab

- Understand ARMv8-A virtual memory system architecture.
- Understand how the kernel manages memory for user processes.
- Understand how demand paging works.
- Understand how copy-on-write works.

Skip to main content

Background

Terminology

Translation Levels

Translating a virtual address to a physical address involves levels of translation. ARMv8-A has 2 to 4 levels of translation for different page sizes and the second stage translation for hypervisors. (not used in labs)

We name each level as in Linux. The top-level is the page global directory (PGD) followed by the page upper directory (PUD), page middle directory (PMD), and page table entry(PTE).

Page v.s. Page Frame v.s. Page Table

Page: A chunk of virtual memory pointed by one entry of PTE.

Block: A chunk of virtual memory pointed by one entry of PUD or PMD.

Page frame: A chunk of physical memory.

Page table: A page frame whose entries point to the next level page tables, blocks, or pages. In this documentation, PGD, PUD, PMD, and PTE are all called page tables.

Page’s Descriptor

As mentioned earlier, each entry of a page table points to the next level page table, a block, or a page. The entry is combined with the page frame’s physical address and attributes of the region.

We list the necessary content for you.

Skip to main content

Descriptor’s Format(simplified)

```
Entry of PGD, PUD, PMD which point to a page table

+-----+------------------------------+---------+--+
|     | next level table's phys addr | ignored |11|
+-----+------------------------------+---------+--+
     47                             12         2  0

Entry of PUD, PMD which point to a block

+-----+------------------------------+---------+--+
|     |  block's physical address    |attribute|01|
+-----+------------------------------+---------+--+
     47                              n         2  0

Entry of PTE which points to a page

+-----+------------------------------+---------+--+
|     |  page's physical address     |attribute|11|
+-----+------------------------------+---------+--+
     47                             12         2  0

Invalid entry

+-----+------------------------------+---------+--+
|     |  page's physical address     |attribute|*0|
+-----+------------------------------+---------+--+
     47                             12         2  0
```

Attributes Used in this Lab

Bits[54]

The unprivileged execute-never bit, non-executable page frame for EL0 if set.

Bits[53]

The privileged execute-never bit, non-executable page frame for EL1 if set.

Bits[47:n]:

The physical address the entry point to. Note that the address should be aligned to 2^n Byte.

Bits[10]

The access flag, a page fault is generated if not set.

Skip to main content

Bits[7]

0 for read-write, 1 for read-only.

Bits[6]

0 for only kernel access, 1 for user/kernel access.

Bits[4:2]

The index to MAIR.

Bits[1:0]

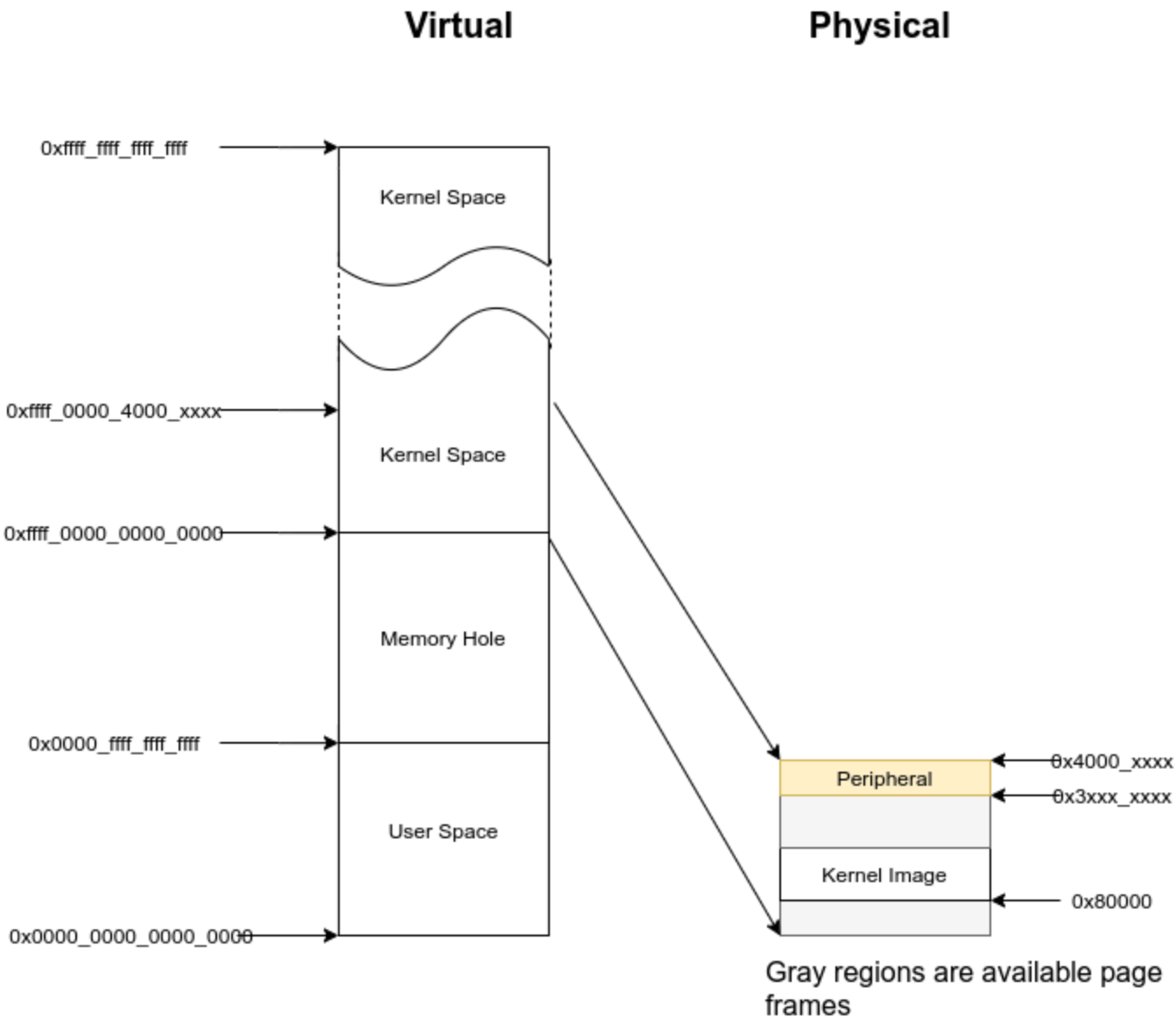
Specify the next level is a block/page, page table, or invalid.

> ⚠ **Warning**
>
> If you set Bits[7:6] to 0b01, which means the user can read/write the region, then the kernel is automatically not executable in that region no matter what the value of Bits[53] is.

AArch64 memory layout

In the 64-bit virtual memory system, the upper address space is usually for kernel mode, and the lower address space is for user mode.

Skip to main content



Note

The entire accessible physical address could be linearly mapped to offset 0xffff_0000_0000_0000 for kernel access in the labs. It simplifies the design.

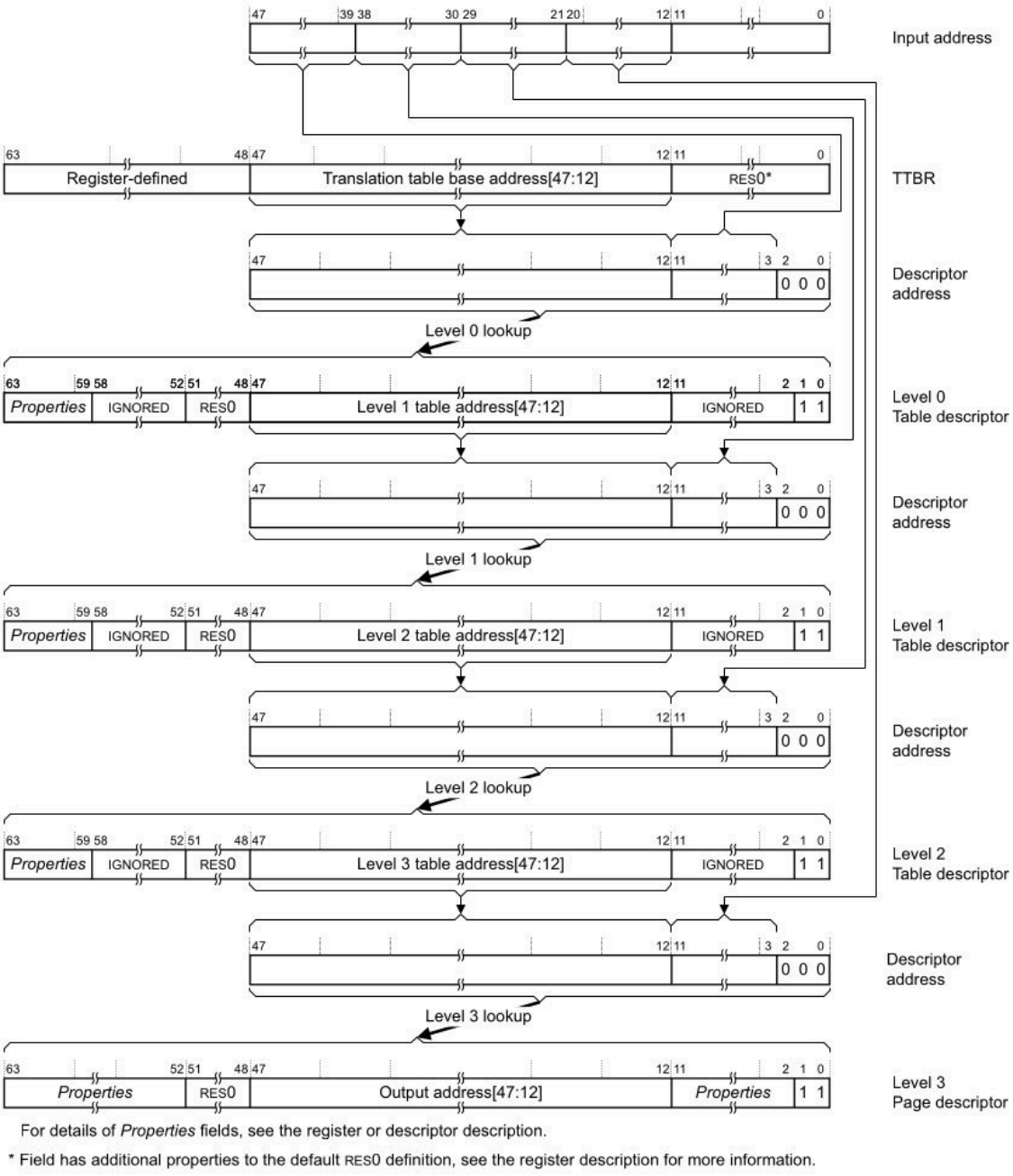
Configuration

ARMv8-A has the elasticity for different configurations. You can change the granularity of paging, the addressable region, etc. To keep everything simple, the following configuration is specified for this lab.

- Disable instruction cache.

Skip to main content

- The addressable region is 48 bit.
- The page granule size is 4KB.
- Not use address space ID (ASID).



Reference

So far, we have briefly introduced the concept of virtual memory and ARMv8-A virtual memory system architecture. For details, you can refer to

Skip to main content

- **The AArch64 Virtual Memory System Architecture Chapter(page 1720)** of ARMv8-A Architecture Reference

Basic Exercises

Basic Exercise 1 - Virtual Memory in Kernel Space - 10%

We provide a step-by-step tutorial to guide you to make your original kernel works with virtual memory. However, we only give the essential explanation in each step. For details, please refer to the manual.

Translation Control Register (TCR)

Paging is configured by TCR. The following basic configuration is used in this lab.

```
#define TCR_CONFIG_REGION_48bit (((64 - 48) << 0) | ((64 - 48) << 16))
#define TCR_CONFIG_4KB ((0b00 << 14) |  (0b10 << 30))
#define TCR_CONFIG_DEFAULT (TCR_CONFIG_REGION_48bit | TCR_CONFIG_4KB)

ldr x0, = TCR_CONFIG_DEFAULT
msr tcr_el1, x0
```

Todo

Set up TCR_EL1.

Memory Attribute Indirection Register (MAIR)

Brief Introduction

The MMU has different memory **access policies** for different **memory regions**.

- Memory **access policies** are encoded as attributes and stored in MAIR.

Skip to main content

- To select the attribute for a certain **memory region**, each page table's entry contains the index to the attribute. (see Attributes Used in this Lab)

When the MMU gets a virtual address, it gets the index from the page table's entry and looks up MAIR to get the memory attribute. Then, it accesses the memory with different access policies.

Used Memory Attributes

The following two attributes are used in the lab.

- Device memory nGnRnE:
  - Peripheral access.
  - The most restricted memory access.
- Normal memory without cache:
  - Normal RAM access.
  - Memory gathering, reordering, and speculative execution are possible but without cache.

```
#define MAIR_DEVICE_nGnRnE 0b00000000
#define MAIR_NORMAL_NOCACHE 0b01000100
#define MAIR_IDX_DEVICE_nGnRnE 0
#define MAIR_IDX_NORMAL_NOCACHE 1

ldr x0, =( \
  (MAIR_DEVICE_nGnRnE << (MAIR_IDX_DEVICE_nGnRnE * 8)) | \
  (MAIR_NORMAL_NOCACHE << (MAIR_IDX_NORMAL_NOCACHE * 8)) \
)
msr mair_el1, x0
```

Todo

Set up `mair_el1`.

Identity Paging

Before enabling the MMU, you need to set up the page tables for the kernel. You can start from

Skip to main content

In a two-level translation, you only need PGD and PUD. Each entry of PUD points to a 1GB block. Hence, you only need

- The first entry of PGD which points to PUD
- The first two entries of PUD.
  - The first one maps 0x00000000 - 0x3fffffff (RAM and GPU peripherals)
  - The second one maps 0x40000000 - 0x7fffffff(ARM local peripherals).

setup

- 2 page frames for PGD and PUD.
- PUD's entries are blocks.
- Map all memory as Device nGnRnE.

```
#define PD_TABLE 0b11
#define PD_BLOCK 0b01
#define PD_ACCESS (1 << 10)
#define BOOT_PGD_ATTR PD_TABLE
#define BOOT_PUD_ATTR (PD_ACCESS | (MAIR_IDX_DEVICE_nGnRnE << 2) | PD_BLOCK)

mov x0, 0 // PGD's page frame at 0x0
mov x1, 0x1000 // PUD's page frame at 0x1000

ldr x2, = BOOT_PGD_ATTR
orr x2, x1, x2 // combine the physical address of next level page with attribut
str x2, [x0]

ldr x2, = BOOT_PUD_ATTR
mov x3, 0x00000000
orr x3, x2, x3
str x3, [x1] // 1st 1GB mapped by the 1st entry of PUD
mov x3, 0x40000000
orr x3, x2, x3
str x3, [x1, 8] // 2nd 1GB mapped by the 2nd entry of PUD

msr ttbr0_el1, x0 // load PGD to the bottom translation-based register.

mrs x2, sctlr_el1
orr x2 , x2, 1
msr sctlr_el1, x2 // enable MMU, cache remains disabled
```

If you setup correctly, your kernel should work as before.

Skip to main content

Todo

Set up identity paging.

Map the Kernel Space

As mentioned earlier, the kernel space is the upper address space. Now, you need to modify your linker script to make your kernel's symbols in the upper address space.

```
SECTIONS
{
  . = 0xffff000000000000; // kernel space
  . += 0x80000; // kernel load address
  _kernel_start = . ;
  // ...
}
```

After the kernel is re-built and loaded, load the identity paging's PGD to `ttbr1_el1`. Next, enable the MMU and use an indirect branch to the virtual address. Then, the CPU is running your kernel in the upper address space.

```
// ...

msr ttbr0_el1, x0
msr ttbr1_el1, x0 // also load PGD to the upper translation based register.
mrs x2, sctlr_el1
orr x2 , x2, 1
msr sctlr_el1, x2

ldr x2, = boot_rest // indirect branch to the virtual address
br x2

boot_rest:
// ...
```

Todo

Modify the linker script, and map the kernel space.

Skip to main content

Note

If there is a hard-coded address(e.g. IO address) in your kernel, you should also set it to the upper address space.

Finer Granularity Paging

The granularity of two-level translation is 1GB. In the previous setting, all memory regions are mapped as device memory.

However, unaligned access to device memory causes alignment exceptions and the compiler sometimes generates unaligned access. Hence, you should map most of the RAM as normal memory and MMIO region as device memory.

Then, you should use three-level translation(2MB) or four-level translation(4KB) for linear mapping.

Todo

Linear map kernel with finer granularity and map RAM as normal memory.

Basic Exercise 2 - Virtual Memory in User Space - 30%

PGD Allocation

To isolate user processes, you should create an address space for each of them. Hence, the kernel should allocate one PGD for each process when it creates a process.

Map the User Space

Same as kernel space mapping, you need to iteratively fill in the entries of page tables from PGD -> PUD -> PMD -> PTE.

Skip to main content

In this lab, we recommand you to use a iteration-based method to implement the page mapping function. During this process, the next level page tables such as PUD, PMD, and PTE may not already present. You should allocate one page frame to be used as the next level page table. Then, fill the page frame's entries to map the virtual address.

Here is the brief example:

```
pte *walk(pagetable pagetable, uint64_t va, int alloc) {
  for (int level = 3; level > 0; level--) {
    pte *pte = ...;
    if (*pte is a entry) {
      pagetable = *pte;
    } else {
      pagetable = alloc_page();
      memset(pagetable, 0, 0x1000);
      *pte = ...;
    }
  }
  return ...;
}
```

Todo

Implement function like `mappages(pagetable pagetable, uint64_t va, uint64_t size, uint64_t pa, ...)` and use it to map user's code at 0x0 and stack regions at 0xfffffffb000 ~ 0xfffffffff000(4 pages).

Note

You should use 4KB pages for user processes in this lab, so you need PGD, PUD, PMD, and PTE for four-layer translation.

Revisit Syscalls

In lab 5, different user programs used different linker scripts to prevent address overlapping. Also, the child process can't use the same user stack address as the parent.

With virtual memory, the same virtual address can be mapped to different physical addresses. Therefore, you can revisit `fork()`, `exec()` ans `mbox_call` with virtual memory to solve the

Skip to main content

Todo

Revisit syscalls to map the same virtual address to different physical addresses for different processes.

Context Switch

To switch between different address spaces, you can set the translation based register(`ttbr0_el1`) with different PGDs.

In addition, you might need memory barriers to guarantee previous instructions are finished. Also, a TLB invalidation is needed because the old values are staled.

```
ldr x0, = next_pgd
dsb ish // ensure write has completed
msr ttbr0_el1, x0 // switch translation based address.
tlbi vmalle1is // invalidate all TLB entries
dsb ish // ensure completion of TLB invalidatation
isb // clear pipeline
```

Todo

Set `ttbr0_el1` to switch the address space in context switches.

Video Player - 40%

In order to test the correctness of your previous implementation, we create a `user program` that runs only if your kernel behaves as expected.

Note

The user program uses all syscalls in previous lab.

Skip to main content

> **Warning**
>
> Only if you can run our test program fluently will you receive all the points; otherwise, even though you implemented the system call correctly, you will receive no points in this section.

Advanced Exercises

Advanced Exercise 1 - Mmap - 10%

`mmap()` is the system call to create memory regions for a user process. Each region can be mapped to a file or anonymous page(the page frames not related to any file) with different protection. Then, users can create heap and memory-mapped files using the system call.

Besides, the kernel can also use it for implementing the program loader. Memory regions such as .text and .data can be created by **memory-mapped files**. Memory regions such as **.bss** and **user stack** can be created by **anonymous page mapping**.

> **Note**
>
> Because we don't have an ELF file for memory-mapped files, you only need to implement the anonymous page mapping in this Lab.

API Specification

(void*) mmap(void* addr, size_t len, int prot, int flags, int fd, int file_offset)

The kernel uses **addr** and **len** to create a new valid region for the current process.

- If addr is NULL, the kernel decides the new region's start address
- If addr is not NULL
  - If the new region overlaps with existing regions, or addr is not page-aligned, the kernel takes addr as a hint and decides the new region's start address.
  - Otherwise, the kernel uses addr as the new region's start address.

Skip to main content

- The memory region created by mmap() should be page-aligned, if the len is not multiple of the page size, the kernel rounds it up.

prot is the region's access protection

> - PROT_NONE : 0, not accessible
> - PROT_READ : 1, readable
> - PROT_WRITE : 2, writable
> - PROT_EXEC : 4, executable

The following **flags** should be implemented

> - MAP_ANONYMOUS: New region is mapped to anonymous page. It's usually used for stack and heap.
> - MAP_POPULATE: After `mmap()`, it directly does region_map. (You don't have to implement it if you implement demand paging)

Region Page Mapping

If the user specifies MAP_POPULATE in the mmap() call. The kernel should create the page mapping for the newly created region.

- If the region is mapped to anonymous pages
  > 1. Allocate page frames.
  > 2. Map the region to page frames, and set the page attributes according to region's protection policy.

Todo

Implement system call mmap, syscall number: 10.

Skip to main content

Advanced Exercise 2 - Page Fault Handler & Demand Paging - 10%

The page frames are pre-allocated in the previous parts. However, user program might allocate a huge space on heap or memory mapped files without using it. The kerenl wastes the CPU time and the physical memory on this things.

In this part, your kernel should allocate page frames for user processes on demand. The kernel only allocates the PGD for newly created process in the beggining.

When a page fault is generated,

- If the fault address is not part of any region in the process's address space,
  - A segmentation fault is generated, and the kernel terminates the process.
- If it's part of one region,

  Follow region_map but only map **one page frame**. for the fault address.

> **Note**
>
> To justify the correctness, you should print the log of each fault; here's an brief example.

```
// For Translation fault
printf("[Translation fault]: %x\n", addr);
// For Segmentation fault
printf("[Segmentation fault]: Kill Process\n");
```

> **Todo**
>
> Implement demand paging.

Advanced Exercise 3 - Copy on Write - 10%

When a process call `fork()` to create a child process, the kernel needs to copy all the page frames owned by the parent in the previous implementation. Otherwise, a write by either child

Skip to main content

However, an `exec()` followed by a `fork()` call is quite common in UNIX programming. The original mapping of child would be destoryed and you waste a lot of time on copying never used page frames. Hence, a copy-on-write mechanism comes to help these odds.

The following statements is a possible copy-on-write implementation.

On Fork a New Process

1. Copy the page frames of page tables.
2. Then mark PTE entries of **both child and parent** to be **read-only** even for original read-write pages.

When Either Children or Parent Write to that Page

A permission fault is generated because the PTE entry marks as read-only, then you should

Check the region's permission in the address space.

- If the corresponding region is **read-only**, then the **segmentation fault** is generated because the user trying to write a read-only region.
- If the corresponding region is **read-write**, then it's a **copy-on-write fault**.
  - The kernel should allocate a page frame, copy the data, and modify the table's entry to be correct permission.

> **Note**
>
> `fork()` may be executed many times, so page frames may be shared by many children and one parent. Hence, you need a reference count for each page frame. And you should not reclaim the page frame if there is still someone referring to it.

> **Warning**
>
> Remember to remap the chlid process' mailbox memory in fork(); our user program should still work under the copy-on-write policy.

Skip to main content

Todo

Implement copy-on-write.

Previous
Lab 5: Thread and User Process

Next
Lab 7: Virtual File System